COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	September 21, 2026	**DATE SUBMITTED:**	September 21, 2026
NAME:	Mendoza, Ryo Shin P.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 9.1 TREE ADT

1. PROCEDURE OUTPUT

ILO A: Create C++ code to implement a general tree, binary tree and binary search tree.

Task 1: Create code in C++ that will create a tree as shown in the figure above. Use linked lists as the internal representation of this tree. Indicate your code screenshot and comments in table 9-1.

```cpp
#include <iostream>
#include "generalTree.h"

int main()
{
    // Create the root node with the value 'A'
    GeneralTree<char> *root = new GeneralTree<char>('A');

    // Create the children of root A
    root->insertChild('B');
    root->insertChild('C');
    root->insertChild('D');
    root->insertChild('E');
    root->insertChild('F');
    root->insertChild('G');

    // Access node D and create its child H
    root->getChild(2)->insertChild('H');

    // Access node E and create its children I and J
    root->getChild(3)->insertChild('I');
    root->getChild(3)->insertChild('J');

    // Access node J and create its children P and Q
    root->getChild(3)->getChild(1)->insertChild('P');
    root->getChild(3)->getChild(1)->insertChild('Q');

    // Access node F and create its children K, L, and M
    root->getChild(4)->insertChild('K');
    root->getChild(4)->insertChild('L');
    root->getChild(4)->insertChild('M');

    // Access node G and create its child N
    root->getChild(5)->insertChild('N');

    // Display the value of the root node
    std::cout << "Root: "
              << root->getRootVal() << std::endl;

    // Display the first child of A
    std::cout << "First Child of A: "
              << root->getChild(0)->getRootVal() << std::endl;

    // Display the third child of A
    std::cout << "Third Child of A: "
              << root->getChild(2)->getRootVal() << std::endl;
```

```
    // Display the child of D
    std::cout << "Child of D: "
              << root->getChild(2)->getChild(0)->getRootVal()
              << std::endl;

    // Display the second child of E
    std::cout << "Second Child of E: "
              << root->getChild(3)->getChild(1)->getRootVal()
              << std::endl;

    // Display the first child of J
    std::cout << "First Child of J: "
              << root->getChild(3)
                   ->getChild(1)
                   ->getChild(0)
                   ->getRootVal()
              << std::endl;
```

Table 9-1. Main Code

```
#ifndef GENERALTREE_H
#define GENERALTREE_H

// GeneralTree is a template class, so it can store different data types
template <typename T>
class GeneralTree
{
private:
    // Stores the value/data of the current node
    T key;

    // Pointer to the first child of the current node
    GeneralTree<T> *firstChild;

    // Pointer to the next sibling of the current node
    GeneralTree<T> *nextSibling;

public:
    // Constructor that creates a new node with the given value
    GeneralTree(T rootnode);

    // Function used to insert a new child node
    void insertChild(T newNode);

    // Function used to access a child node
    GeneralTree<T> *getChild(int index);
```

```
    // Functions used to get and change the value of a node
    void setRootVal(T obj);
    T getRootVal();

    // Checks whether the current node is a leaf
    bool isLeaf();
    // Functions used for tree traversals
    void preOrder();
    void postOrder();
    void inOrder();

    bool findData(int choice, T key);
};

// Constructor
template <typename T>
GeneralTree<T>::GeneralTree(T rootobj)
{
    // Assign the given value to the node
    key = rootobj;

    // Set the first child and next sibling to nullptr
    // because the new node has no children or siblings yet
    firstChild = nullptr;
    nextSibling = nullptr;
}
```

```
// Insert a new child node
template <typename T>
void GeneralTree<T>::insertChild(T newNode)
{
    // Create a new node with the given value
    GeneralTree<T> *newChild = new GeneralTree<T>(newNode);

    // Check if the current node has no children
    if (firstChild == nullptr)
    {
        // Make the new node the first child
        firstChild = newChild;
    }
    else
    {
        // Start at the first child
        GeneralTree<T> *current = firstChild;

        // Move through the sibling list
        // until the last sibling is reached
        while (current->nextSibling != nullptr)
        {
            current = current->nextSibling;
        }

        // Add the new node at the end of the sibling list
        current->nextSibling = newChild;
    }
}
```

```cpp
// Access a child using its index
template <typename T>
GeneralTree<T> *GeneralTree<T>::getChild(int index)
{
    // Start at the first child
    GeneralTree<T> *current = firstChild;

    // Move through the siblings until the requested index
    for (int i = 0; i < index && current != nullptr; i++)
    {
        current = current->nextSibling;
    }

    // Return the child node
    return current;
}

// Change the value of the current node
template <typename T>
void GeneralTree<T>::setRootVal(T obj)
{
    key = obj;
}

// Get the value of the current node
template <typename T>
T GeneralTree<T>::getRootVal()
{
    return key;
}

// Check if the current node is a leaf node
template <typename T>
bool GeneralTree<T>::isLeaf()
{
    // A node is a leaf if it has no children
    return firstChild == nullptr;
}
```

Table 9-1-1. Header Code

```
Root: A
First Child of A: B
Third Child of A: D
Child of D: H
Second Child of E: J
First Child of J: P
```

Table 9-1-2. Output

Task 2: Complete the following table:

Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

Table 9-2

ILO B: Create C++ code for implementation of tree traversal methods such as pre-order, in-order and post-order traversal.

Task 3: After implementing the code for above, answer the following:
3.1 Given the tree diagram, find the result of the pre-order, post-order and in-order traversal strategies by hand. Include this as table 9-3 in section 6.

Pre-Order	A B C D H E I J P Q F K L M G N
Post-Order	B C H D I P Q J E K L M F N G A

In-Order	B C H D I P Q J E K L M F N G A

Table 9-3

3.2 Create a function for pre-order, post-order and in-order traversal. Make sure that each function displays an output into the console. Once you have the output, create and fill table 9-4 in section 6 so that it contains the screenshot of the function, screenshot of the output and your observations. Your observations consist of a comparison between output in #1 and the output of your functions in #2.

```
template <typename T>
void GeneralTree<T>::preOrder()
{
    // Display the current node first
    std::cout << key << " ";

    // Traverse the first child
    if (firstChild != nullptr)
    {
        firstChild->preOrder();
    }

    // Traverse the next sibling
    if (nextSibling != nullptr)
    {
        nextSibling->preOrder();
    }
}

template <typename T>
void GeneralTree<T>::postOrder()
{
    // Traverse the children first
    if (firstChild != nullptr)
    {
        firstChild->postOrder();
    }

    // Display the current node
    std::cout << key << " ";

    // Traverse the next sibling
    if (nextSibling != nullptr)
    {
        nextSibling->postOrder();
    }
```

```cpp
template <typename T>
void GeneralTree<T>::inOrder()
{
    // Traverse the first child first
    if (firstChild != nullptr)
    {
        firstChild->inOrder();
    }

    // Display the current node
    std::cout << key << " ";

    // Traverse the next sibling
    if (nextSibling != nullptr)
    {
        nextSibling->inOrder();
    }
}
```

```cpp
    // Display the pre-order traversal
    std::cout << "\nPre-order: ";
    root->preOrder();

    // Display the post-order traversal
    std::cout << "\nPost-order: ";
    root->postOrder();

    // Display the in-order traversal
    std::cout << "\nIn-order: ";
    root->inOrder();
```

```
Pre-order: A B C D H E I J P Q F K L M G N
Post-order: B C H D I P Q J E K L M F N G A
In-order: B C H D I P Q J E K L M F N G A
```

Table 9-4

3.3. Create a function called findData that will take the following parameter: “CHOICE, KEY”. Choice will determine which traversal method will be used to traverse the tree to find the second parameter KEY. If the KEY is found, the value will be displayed such that: “{KEY} was found!”. Otherwise, no output. Include your output as table 9-5 in section 6.

```cpp
template <typename T>
bool GeneralTree<T>::findData(int choice, T searchKey)
{
    bool found = false;

    // Pre-order
    if (choice == 1)
    {
        if (key == searchKey)
        {
            std::cout << "{" << searchKey << "} was found!" << std::endl;
            return true;
        }

        if (firstChild != nullptr)
        {
            found = firstChild->findData(choice, searchKey);

            if (found)
            {
                return true;
            }
        }

        if (nextSibling != nullptr)
        {
            found = nextSibling->findData(choice, searchKey);

            if (found)
            {
                return true;
            }
        }
    }
```

```
// Post-order
else if (choice == 2)
{
    if (firstChild != nullptr)
    {
        found = firstChild->findData(choice, searchKey);

        if (found)
        {
            return true;
        }
    }

    if (nextSibling != nullptr)
    {
        found = nextSibling->findData(choice, searchKey);

        if (found)
        {
            return true;
        }
    }

    if (key == searchKey)
    {
        std::cout << "{" << searchKey << "} was found!" << std::endl;
        return true;
    }
}
```

```cpp
// In-order
else if (choice == 3)
{
    if (firstChild != nullptr)
    {
        found = firstChild->findData(choice, searchKey);

        if (found)
        {
            return true;
        }
    }

    if (key == searchKey)
    {
        std::cout << "{" << searchKey << "} was found!" << std::endl;
        return true;
    }

    if (nextSibling != nullptr)
    {
        found = nextSibling->findData(choice, searchKey);

        if (found)
        {
            return true;
        }
    }
}

return false;
```

```cpp
// Search for J using pre-order
root->findData(1, 'J');

// Search for P using post-order
root->findData(2, 'P');

// Search for Z using in-order
root->findData(3, 'Z');
```

```
{J} was found!
{P} was found!
```

Table 9-5

3.4. Add a new leaf node called which will contain the data value of “O”. Make this node the child of nodeG. Perform the findData function you created in #3, what can you observe about the output? Include your output and answers as table 9-6 in section 6.

```
// Add O as a new child of G
root->getChild(5)->insertChild('O');

// Search for O using pre-order
root->findData(1, 'O');
```

```
{O} was found!
```

Table 9-6

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve given problems using the tree data structure's implementation in C++

Main Code

```cpp
#include <iostream>
#include "bst.h"

int main()
{
    // Create the root node with the value 2
    BinarySearchTree<int> *root =
        new BinarySearchTree<int>(2);

    // Insert the remaining values
    root->insert(3);
    root->insert(9);
    root->insert(18);
    root->insert(0);
    root->insert(1);
    root->insert(4);
    root->insert(5);

    // Display the root
    std::cout << "Root: "
              << root->getRootVal() << std::endl;

    std::cout << "In-order Traversal: ";
    root->inOrder();

    std::cout << "\nPre-order Traversal: ";
    root->preOrder();

    std::cout << "\nPost-order Traversal: ";
    root->postOrder();

    std::cout << std::endl;

    return 0;
}
```

Header Code

```
#ifndef BST_H
#define BST_H

// BinarySearchTree is a template class
template <typename T>
class BinarySearchTree
{
private:
    // Stores the value of the current node
    T key;

    // Pointer to the left child
    BinarySearchTree<T> *leftChild;

    // Pointer to the right child
    BinarySearchTree<T> *rightChild;

public:
    // Constructor
    BinarySearchTree(T rootnode);

    // Insert a new value into the BST
    void insert(T newNode);

    // Access the left and right children
    BinarySearchTree<T> *getLeftChild();
    BinarySearchTree<T> *getRightChild();

    // Get the value of the current node
    T getRootVal();
```

```
    // Traversal functions
    void inOrder();
    void preOrder();
    void postOrder();
};


// Constructor
template <typename T>
BinarySearchTree<T>::BinarySearchTree(T rootnode)
{
    // Assign the given value to the node
    key = rootnode;

    // Set the children to nullptr
    leftChild = nullptr;
    rightChild = nullptr;
}
```

```cpp
// Insert a value into the Binary Search Tree
template <typename T>
void BinarySearchTree<T>::insert(T newNode)
{
    // Smaller values go to the left
    if (newNode < key)
    {
        if (leftChild == nullptr)
        {
            leftChild = new BinarySearchTree<T>(newNode);
        }
        else
        {
            leftChild->insert(newNode);
        }
    }

    // Larger values go to the right
    else if (newNode > key)
    {
        if (rightChild == nullptr)
        {
            rightChild = new BinarySearchTree<T>(newNode);
        }
        else
        {
            rightChild->insert(newNode);
        }
    }
}
```

```cpp
// Get the left child
template <typename T>
BinarySearchTree<T> *BinarySearchTree<T>::getLeftChild()
{
    return leftChild;
}


// Get the right child
template <typename T>
BinarySearchTree<T> *BinarySearchTree<T>::getRightChild()
{
    return rightChild;
}


// Get the value of the current node
template <typename T>
T BinarySearchTree<T>::getRootVal()
{
    return key;
}
```

```cpp
// In-order traversal
template <typename T>
void BinarySearchTree<T>::inOrder()
{
    if (leftChild != nullptr)
    {
        leftChild->inOrder();
    }

    std::cout << key << " ";

    if (rightChild != nullptr)
    {
        rightChild->inOrder();
    }
}


// Pre-order traversal
template <typename T>
void BinarySearchTree<T>::preOrder()
{
    std::cout << key << " ";

    if (leftChild != nullptr)
    {
        leftChild->preOrder();
    }

    if (rightChild != nullptr)
    {
        rightChild->preOrder();
    }
}
```

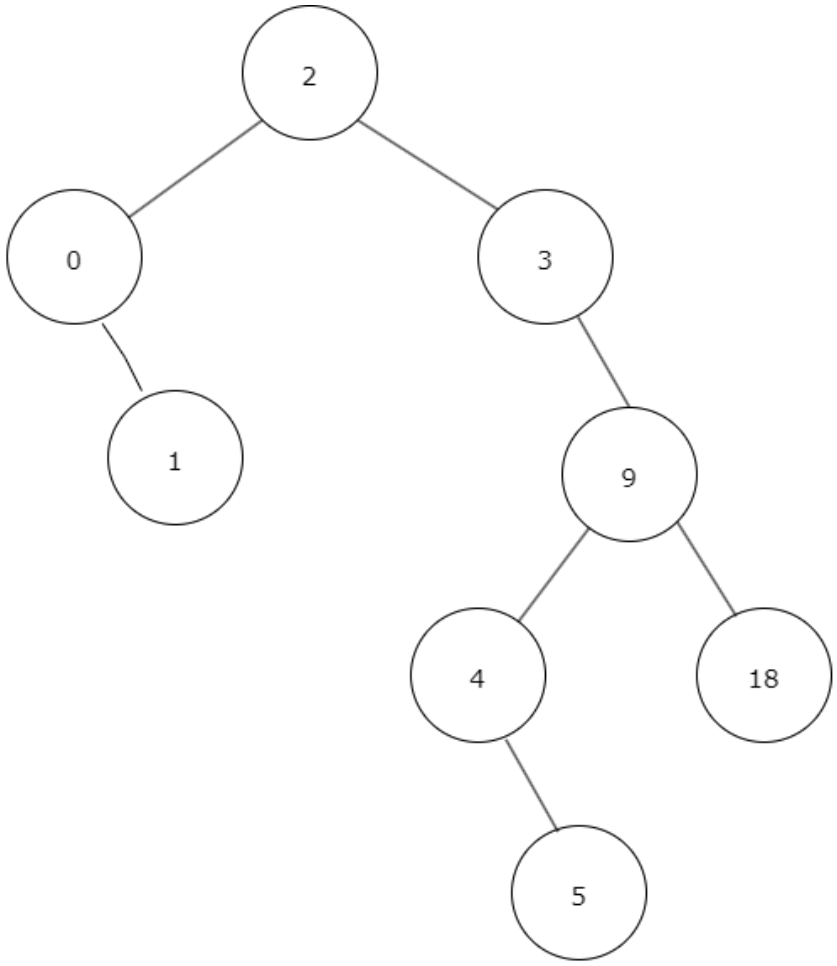
```cpp
// Post-order traversal
template <typename T>
void BinarySearchTree<T>::postOrder()
{
    if (leftChild != nullptr)
    {
        leftChild->postOrder();
    }

    if (rightChild != nullptr)
    {
        rightChild->postOrder();
    }

    std::cout << key << " ";
}

#endif
```

Tree Diagram



In-Order:

0, 1, 2, 3, 4, 5, 9, 18

Post-Order:

1, 0, 5, 4, 18, 9, 3, 2

Pre-Order:

2, 0, 1, 3, 9, 4, 5, 18

Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

```
std::cout << "In-order Traversal: ";
root->inOrder();
```

Given the same input values above, what is the output with different traversal methods? For each output below, indicate your observation: is the output different from the pre-order and post-order traversal that you indicated in the diagrams shown in item #2.

Pre-order Traversal

```
// Pre-order traversal
template <typename T>
void BinarySearchTree<T>::preOrder()
{
    std::cout << key << " ";

    if (leftChild != nullptr)
    {
        leftChild->preOrder();
    }

    if (rightChild != nullptr)
    {
        rightChild->preOrder();
    }
}
```

```
Pre-order Traversal: 2 0 1 3 9 4 5 18
```

Observation: The output is the same as the pre-order traversal indicated in the diagram in Item #2. It follows **Root → Left → Right**.

Post-Order Traversal

```
// Post-order traversal
template <typename T>
void BinarySearchTree<T>::postOrder()
{
    if (leftChild != nullptr)
    {
        leftChild->postOrder();
    }

    if (rightChild != nullptr)
    {
        rightChild->postOrder();
    }

    std::cout << key << " ";
}
```

```
Post-order Traversal: 1 0 5 4 18 9 3 2
```

Observation: The output is the same as the post-order traversal indicated in the diagram in Item #2. It follows **Left** → **Right** → **Root**.

In-Order Traversal

```
// In-order traversal
template <typename T>
void BinarySearchTree<T>::inOrder()
{
    if (leftChild != nullptr)
    {
        leftChild->inOrder();
    }

    std::cout << key << " ";

    if (rightChild != nullptr)
    {
        rightChild->inOrder();
    }
}
```

```
In-order Traversal: 0 1 2 3 4 5 9 18
```

Observation: The output is different from both the pre-order and post-order traversal. In-order follows **Left** → **Root** → **Right**, which produces the values in ascending order because the tree is a Binary Search Tree.

Conclusion
In this activity, I learned how to create and use a Binary Search Tree in C++. I also learned that different traversal methods produce different orders of the same nodes. In-order traversal arranged the values from smallest to largest, while pre-order and post-order followed their respective traversal patterns.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Assistance**

COMPUTER ENGINEERING

QUEZON CITY

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I confirm that AI did not replace my critical thinking for this activity.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.